

A PROJECT REPORT ON

AI CROP DISEASE DETECTION AND ADVISORY SYSTEM FOR FARMERS

Domain: Agriculture / Computer Vision

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	31
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	32

CHAPTER I

INTRODUCTION

Agriculture is the backbone of the Indian economy and supports more than half of the country's population. Crops such as paddy, banana and coconut are grown widely in the Cauvery delta and other regions of Tamil Nadu. Every season a large share of the yield is lost because plant diseases such as leaf blast, bacterial leaf blight, Sigatoka leaf spot and bud rot are not identified early enough. By the time the farmer notices the symptoms clearly, the infection has already spread across the field.

Traditionally, a farmer identifies a disease by experience or by asking a neighbour, a pesticide shop owner or an agricultural extension officer. The number of agricultural officers is very small compared with the number of farms, and they cannot visit every field in time. As a result, farmers often apply the wrong pesticide or an excessive dose, which increases the cost of cultivation, harms beneficial insects and damages soil health.

Recent advances in deep learning, particularly Convolutional Neural Networks (CNN), have made it possible to recognise plant diseases from a simple photograph of a leaf with accuracy comparable to human experts. Smartphones with good cameras are now available even in villages. This project uses these two developments to build a mobile-friendly system in which a farmer photographs a diseased leaf and immediately receives the disease name, its confidence level, and treatment advice in Tamil or English, both as text and as voice.

LIMITATIONS

1. Disease identification depends on expert availability, which is limited in rural areas.
2. Advice given by pesticide shops may be biased towards selling more chemicals.
3. Most existing apps provide output only in English and are difficult for farmers with low literacy.
4. There is no central record of where diseases are occurring, so outbreaks are detected late.

1.1 History of Domain

Plant pathology as a science began in the nineteenth century when Anton de Bary proved that the potato late blight that caused the Irish famine was caused by a fungus. For more than a century disease diagnosis remained a manual process based on visual symptoms and laboratory culture of the pathogen.

Computer vision was first applied to agriculture in the 1990s using hand-crafted features such as colour histograms, texture (GLCM) and shape descriptors with classifiers such as SVM. The release of the PlantVillage dataset in 2015, containing more than 54,000 labelled leaf images of 38 crop-disease classes, and the success of deep CNNs on ImageNet led to a rapid growth of deep-learning based disease detection. Lightweight networks such as MobileNet now allow such models to run directly on smartphones.

1.2 Overview of the Project

CropCare AI is a web and mobile-friendly application. The farmer registers with a mobile number, selects the crop, and uploads or captures a photograph of the affected leaf. The image is pre-processed, the leaf region is segmented from the background, and a MobileNetV2 model fine-tuned on PlantVillage and locally collected images classifies the disease. If the confidence is below a threshold, the farmer is asked to take another clearer photo.

The predicted disease is mapped to a treatment knowledge base prepared from Tamil Nadu Agricultural University (TNAU) agritech recommendations. The advice includes organic remedies, chemical remedies with dosage per litre, preventive measures and nearby availability. The result is shown in Tamil and English and can be read aloud. Every diagnosis is logged with its GPS location so that the agriculture department can view an outbreak heat map.

1.3 Motivation

Farmers in delta regions often lose a large part of their harvest simply because a disease was not recognised in its early stage. Expert advice is far away, while a smartphone is

already in the farmer's pocket.

- **Motivation:** To put an expert-level, always-available plant doctor in the hands of every farmer in their own language.
- **Outcome:** Early detection, correct and minimal use of pesticides, lower cultivation cost and real-time outbreak data for officials.

1.4 Aim of the Project

The project is entitled as “**AI CROP DISEASE DETECTION AND ADVISORY SYSTEM FOR FARMERS**”. The aim of this project is to develop an intelligent system that detects crop diseases from leaf images using deep learning and provides instant, reliable and local-language treatment advice to farmers, while collecting location data to help authorities monitor disease outbreaks.

1.5 Objectives

- To build a CNN model that classifies leaf diseases of paddy, banana, coconut, tomato and other crops with high accuracy.
- To reject unclear images using a confidence threshold so that wrong advice is not given.
- To provide organic and chemical treatment recommendations with dosage in Tamil and English.
- To provide text-to-speech output for farmers with low literacy.
- To generate a district-wise outbreak heat map for agricultural officers.

1.6 Literature Review

Hughes and Salathe (2015): Released the PlantVillage dataset, an open-access repository of more than 50,000 expertly curated images of healthy and infected leaves, enabling the development of mobile disease diagnostics.

Mohanty, Hughes and Salathe (2016): Trained AlexNet and GoogLeNet on PlantVillage images of 14 crops and 26 diseases and achieved 99.35% accuracy on a held-out test set,

demonstrating the feasibility of smartphone-assisted disease diagnosis. They also noted that accuracy drops on images taken under field conditions.

Ferentinos (2018): Compared several CNN architectures on 87,848 images including field images and reported a best success rate of 99.53% with the VGG architecture, while stressing the importance of field images in training data.

Sandler et al. (2018): Proposed MobileNetV2 with inverted residuals and linear bottlenecks, which provides good accuracy with very few parameters, making it suitable for mobile and edge deployment.

Too et al. (2019): Performed a comparative study of fine-tuned deep learning models (VGG16, Inception V4, ResNet and DenseNet) for plant disease identification and found DenseNet to give the best accuracy with fewer parameters.

TNAU Agritech Portal: The Tamil Nadu Agricultural University portal provides crop-wise integrated pest and disease management recommendations which are used as the source for the treatment knowledge base of this project.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, farmers identify crop diseases manually based on their own experience or by consulting neighbours, pesticide dealers or agricultural officers. Some farmers send photos through WhatsApp groups and wait for a reply from an expert. A few mobile applications exist, but they work mainly in English, are trained on laboratory images, and do not give locally relevant treatment advice.

DISADVANTAGES

- Diagnosis is delayed, allowing the disease to spread.
- Wrong identification leads to wrong pesticide and excessive dosage.
- Expert advice is not available at all times.
- No language and voice support for low-literacy farmers.
- No centralised data on disease occurrence.

2.2 Proposed System

The proposed system is an AI-based crop disease detection and advisory system. A MobileNetV2 CNN fine-tuned using transfer learning classifies the leaf image within a few seconds. A confidence check prevents unreliable predictions, and a rule-based advisory engine maps each disease to treatment advice. Output is provided in Tamil and English with voice support, and every diagnosis is stored with location for outbreak analysis.

ADVANTAGES

- Instant diagnosis within seconds using a smartphone photo.
- High accuracy using transfer learning on field and laboratory images.
- Advice in Tamil and English with audio output.
- Reduces unnecessary pesticide use and cost.

- Outbreak heat map supports early action by the agriculture department.

2.3 Project Schedule

The development of CropCare AI follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the CropCare AI system are listed below.

- **User registration and login:** Farmers register with name, mobile number, village and preferred language.
- **Image capture and upload:** The system accepts JPG/PNG leaf images from camera or gallery.
- **Disease prediction:** The CNN model predicts the disease class and confidence score.
- **Low confidence handling:** If confidence is below 70%, the user is asked to retake the photo.
- **Advisory generation:** Treatment, dosage and prevention steps are displayed for the predicted disease.
- **Voice output:** The advice can be played as Tamil or English speech.
- **History and heat map:** Users view past diagnoses; officers view an outbreak map.
- **Admin management:** Admin updates crops, diseases and treatment knowledge base.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements

describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Performance:** Prediction must be returned within 3 seconds on a normal 4G connection.
- **Usability:** Large buttons, icons and minimal typing suitable for farmers.
- **Accuracy:** The model should achieve at least 90% accuracy on the test set.
- **Availability:** The service should be available 24x7.
- **Security:** Passwords hashed; location data visible only to authorised officers.
- **Scalability:** Able to serve thousands of farmers during the season.

3.3 User Requirements

- Simple interface with local language support
- Quick result after taking a photo
- Clear treatment steps with dosage
- Audio playback of advice
- Access to previous diagnosis history
- Works on low-cost Android phones

3.4 System Requirements

- Web application accessible through a mobile browser
- Trained CNN model hosted on the server
- Database for users, diagnoses and knowledge base
- Text-to-speech service
- Internet connectivity for upload

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System	: Intel i5 Processor 2.4 GHz or above
Hard Disk	: 256 GB SSD
RAM	: 8 GB (16 GB recommended for training)
GPU	: NVIDIA GPU (optional, or Google Colab for training)
Monitor	: 15" colour monitor
Mobile	: Android smartphone with camera (for farmers)

4.2 Software Specification

Operating System	: Windows 10 / Ubuntu 22.04
Coding Language	: Python 3.10, JavaScript
Deep Learning	: TensorFlow 2.x / Keras
Back End	: Flask
Front End	: React JS
Database	: SQLite (development) / PostgreSQL (production)
Dataset	: PlantVillage + locally collected field images
IDE	: VS Code, Jupyter Notebook

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed CropCare AI system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

The project uses mature open-source technologies: TensorFlow for deep learning, Flask for the API and React for the interface. Pre-trained MobileNetV2 weights reduce training time and data requirements. Training can be performed free of cost on Google Colab GPUs. Hence the project is technically feasible.

ECONOMIC FEASIBILITY

All software used is free and open source. The PlantVillage dataset is publicly available. Hosting the application on a low-cost cloud server or college server is inexpensive. For farmers, the only requirement is a smartphone they already own, so the system is economically feasible.

OPERATIONAL FEASIBILITY

The system is designed for farmers with minimal literacy using icons, Tamil text and voice. Agricultural officers can use the dashboard without training. Since it saves time and money for both groups, it is likely to be accepted, so it is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

TENSORFLOW AND KERAS

TensorFlow is an open-source end-to-end machine learning platform developed by the Google Brain team. It provides a flexible ecosystem of tools, libraries and community resources that lets researchers build and deploy machine learning powered applications on servers, browsers and mobile devices.

Keras is the high-level API of TensorFlow. It offers simple and consistent interfaces for building neural networks layer by layer, which reduces the number of user actions required for common use cases. Pre-trained models such as MobileNetV2, ResNet and EfficientNet are available in keras.applications and can be fine-tuned using transfer learning on a small custom dataset.

- Supports CPU, GPU and TPU execution
- Transfer learning with pre-trained ImageNet models
- TensorFlow Lite for running models on mobile phones
- TensorBoard for visualising training progress

FLASK

Flask is a lightweight WSGI web application framework written in Python. It is classified as a micro-framework because it does not require particular tools or libraries; it

has no database abstraction layer or form validation built in, but extensions can add these features as needed.

Flask provides URL routing, request handling, Jinja2 templating and a development server. Because it is small and simple, it is ideal for exposing a trained machine learning model as a REST API or building a small web application quickly.

- Minimal and easy to learn
- Jinja2 templates for HTML pages
- Large set of extensions (Flask-Login, Flask-SQLAlchemy, Flask-CORS)

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

SQLITE

SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured SQL database engine. Unlike client-server databases, SQLite reads and writes directly to ordinary disk files.

Because it requires no separate server process and no configuration, SQLite is ideal for prototypes, mobile applications and small deployments.

POSTGRESQL

PostgreSQL is a powerful, open-source object-relational database system with over 35 years of active development. It has earned a strong reputation for reliability, feature robustness and performance.

PostgreSQL supports ACID transactions, foreign keys, views, triggers, stored procedures and JSON data types. Its extension system allows additional capabilities such as PostGIS for geographic data and pgvector for similarity search.

- Fully ACID compliant
- Advanced SQL and JSON support
- Extensible with PostGIS and other extensions

TEXT-TO-SPEECH (GTTS / COQUI TTS)

gTTS (Google Text-to-Speech) is a Python library that converts text into spoken audio using Google's TTS service and supports Tamil. Coqui TTS is an open-source deep learning toolkit for offline speech synthesis.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of CropCare AI is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

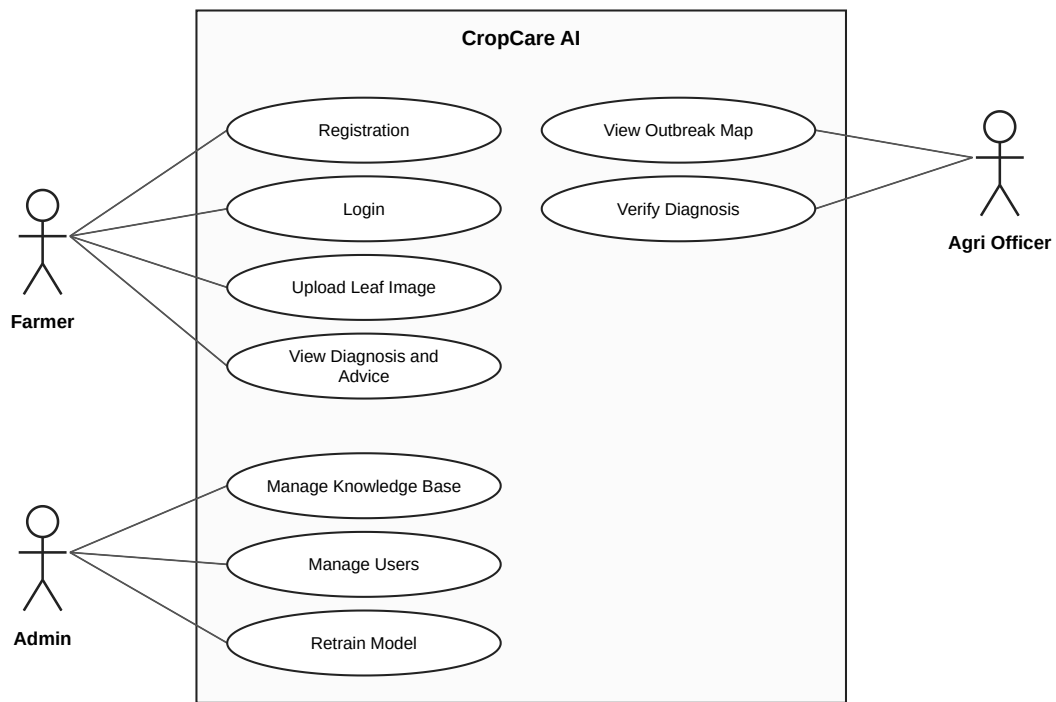


Fig 1: Use case diagram of CropCare AI

6.2 System Architecture

The system follows a three-tier client-server architecture. The presentation layer is a React web application that works on mobile browsers. The application layer is a Flask REST API that performs pre-processing, model inference and advice generation. The data layer stores users, diagnosis logs and the treatment knowledge base.

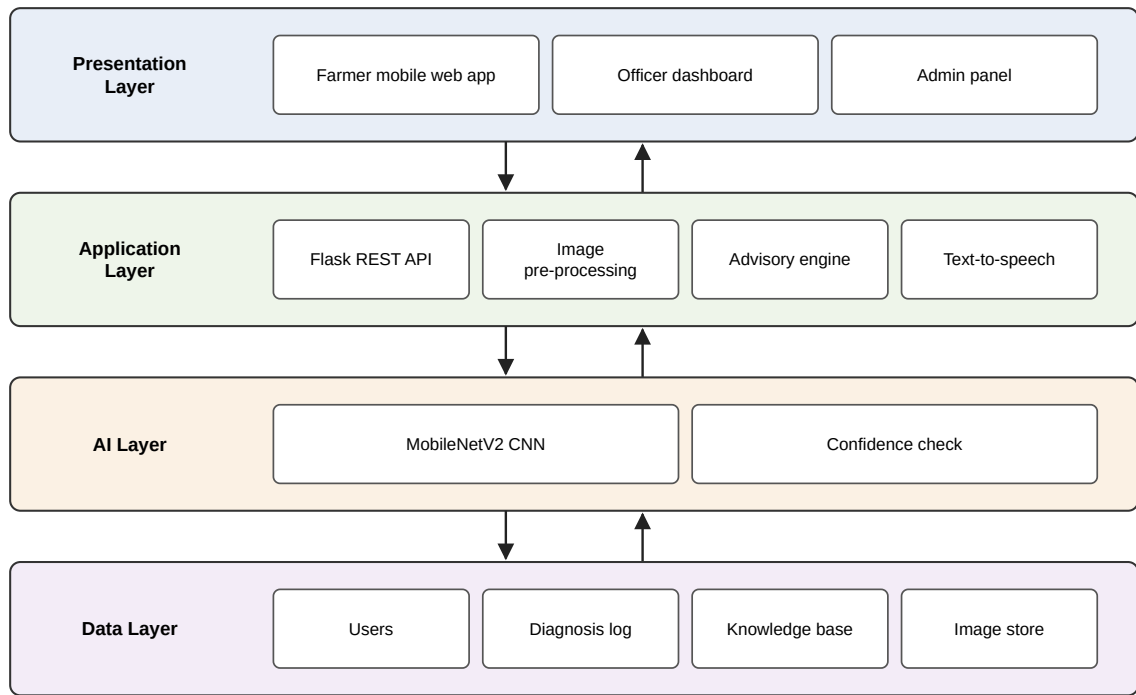


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

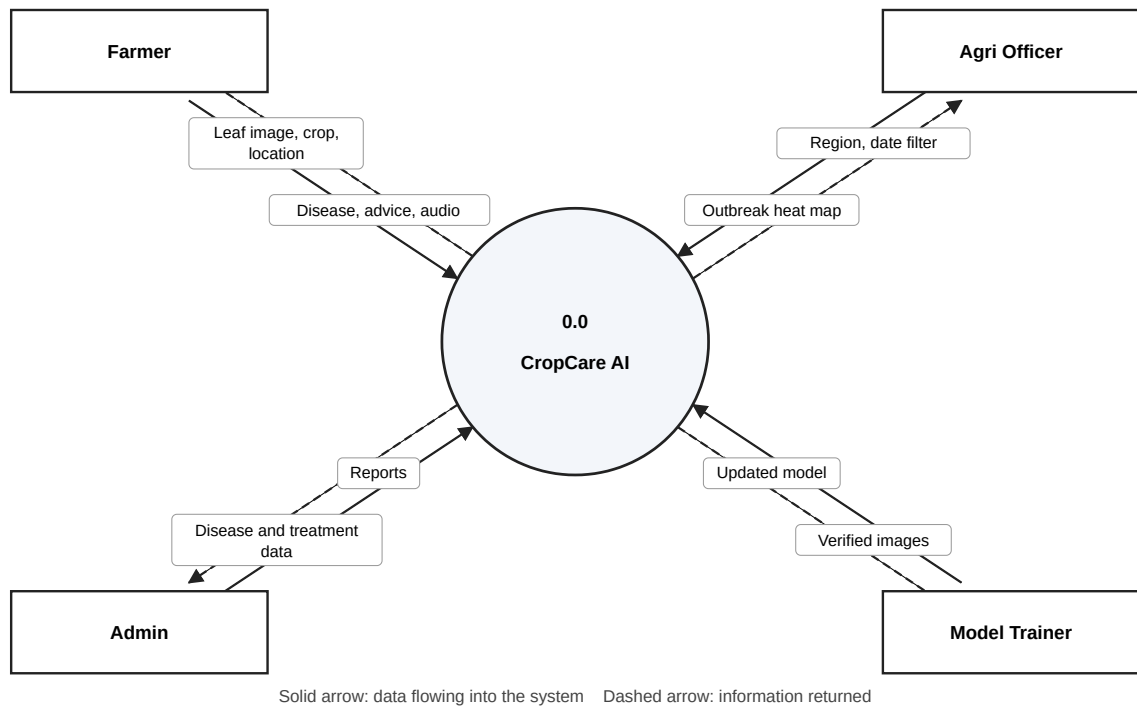


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

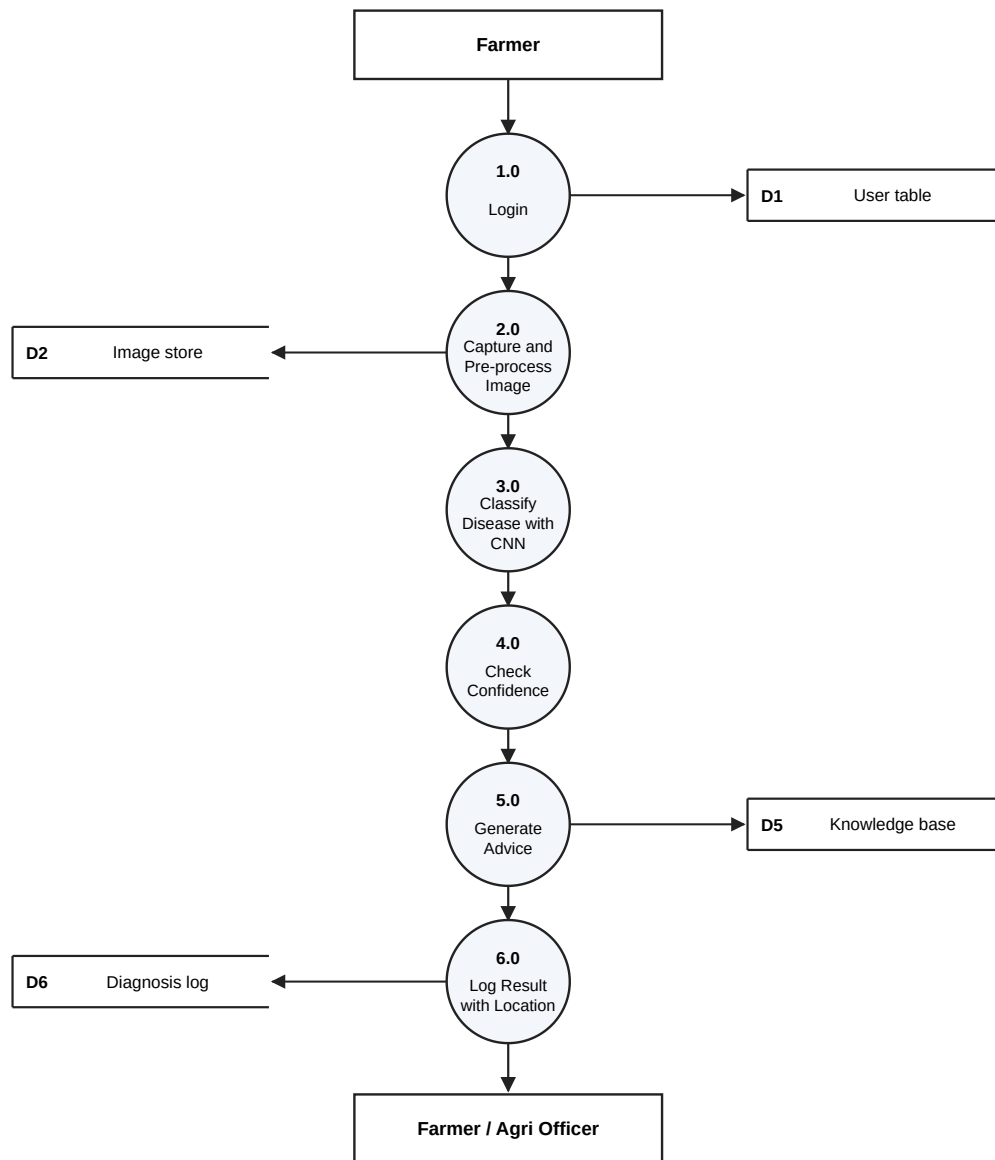


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

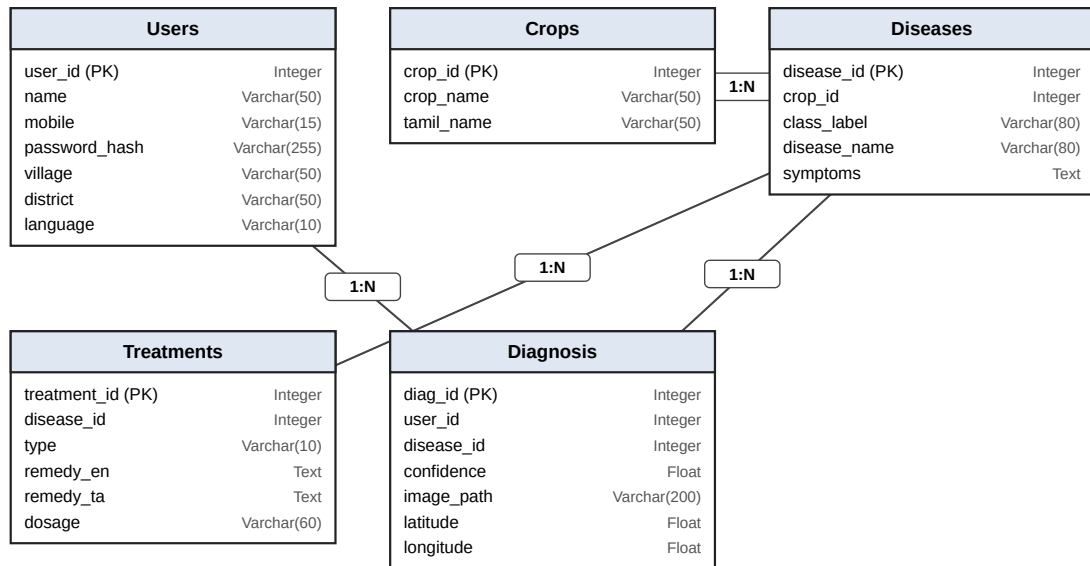


Fig 5: Entity relationship diagram

6.5 Table Design

USERS

FIELD NAME	DATA TYPE	DESCRIPTION
user_id	Integer	Primary key
name	Varchar(50)	Farmer name
mobile	Varchar(15)	Mobile number (login)
password_hash	Varchar(255)	Hashed password
village	Varchar(50)	Village
district	Varchar(50)	District
language	Varchar(10)	ta / en
role	Varchar(10)	farmer / officer / admin

CROPS

FIELD NAME	DATA TYPE	DESCRIPTION
crop_id	Integer	Primary key
crop_name	Varchar(50)	Crop name
tamil_name	Varchar(50)	Name in Tamil

DISEASES

FIELD NAME	DATA TYPE	DESCRIPTION
disease_id	Integer	Primary key
crop_id	Integer	Foreign key to Crops
class_label	Varchar(80)	Model class label
disease_name	Varchar(80)	Display name
symptoms	Text	Symptoms description

TREATMENTS

FIELD NAME	DATA TYPE	DESCRIPTION
treatment_id	Integer	Primary key
disease_id	Integer	Foreign key to Diseases
type	Varchar(10)	organic / chemical
remedy_en	Text	Remedy in English
remedy_ta	Text	Remedy in Tamil
dosage	Varchar(60)	Dosage per litre

DIAGNOSIS

FIELD NAME	DATA TYPE	DESCRIPTION
diag_id	Integer	Primary key
user_id	Integer	Foreign key to Users
disease_id	Integer	Predicted disease
confidence	Float	Model confidence
image_path	Varchar(200)	Stored image
latitude	Float	GPS latitude
longitude	Float	GPS longitude
created_at	Datetime	Date and time

6.6 Algorithm Design

The disease detection uses transfer learning with MobileNetV2. The convolutional base pre-trained on ImageNet extracts features and a new classification head with a softmax layer is trained on leaf images. The algorithm used at prediction time is as follows.

1. **Capture leaf image and resize/normalize** – The image is resized to 224 x 224 pixels and pixel values are scaled to the range [-1, 1] as expected by MobileNetV2.
2. **Remove background using segmentation** – The leaf is separated from soil and background using HSV colour thresholding and the largest contour, reducing noise.
3. **Extract features with a pre-trained CNN (MobileNetV2)** – The convolutional layers produce a 1280-length feature vector through global average pooling.
4. **Classify disease with softmax layer; get confidence score** – A dense softmax layer outputs the probability of each disease class; the highest probability is the confidence.
5. **If confidence < threshold, ask for another photo** – Predictions below 0.70 are rejected to avoid giving wrong advice.
6. **Map disease to treatment knowledge base** – The class label is looked up in the Diseases and Treatments tables.
7. **Return advice in Tamil/English text and voice** – gTTS converts the advice into an MP3 audio clip in the preferred language.
8. **Log result with location for outbreak heat map** – The prediction, GPS coordinates and timestamp are stored for aggregation.

PSEUDO CODE

```
BEGIN
img <- READ uploaded image
img <- RESIZE(img, 224, 224); img <- SEGMENT_LEAF(img)
x <- PREPROCESS(img) // scale to [-1, 1]
p <- MODEL.PREDICT(x) // softmax probabilities
k <- ARGMAX(p); conf <- p[k]
IF conf < 0.70 THEN RETURN "Image unclear - please retake photo"
```

```

disease <- CLASS_LABEL[k]
advice  <- KNOWLEDGE_BASE.LOOKUP(disease, language)
audio   <- TTS(advice, language)
SAVE(user, disease, conf, gps, time)
RETURN disease, conf, advice, audio
END

```

6.7 Modules

The system is divided into modules based on the three types of users: Farmer, Agricultural Officer and Admin. Each module performs a specific function and communicates with the Flask API.

MODULES

FARMER

1. REGISTRATION
2. LOGIN
3. UPLOAD LEAF IMAGE
4. VIEW DIAGNOSIS AND ADVICE

AGRI OFFICER

1. VIEW OUTBREAK MAP
2. VERIFY DIAGNOSIS

ADMIN

1. MANAGE KNOWLEDGE BASE
2. MANAGE USERS
3. RETRAIN MODEL

FARMER

REGISTRATION

In this module the farmer registers by entering name, mobile number, village, district, main crop and preferred language. The details are stored in the user table. Only registered users can store their diagnosis history.

LOGIN

This module provides safe and secure access. Users enter their mobile number and password. Passwords are verified against their hashed values; if unauthenticated users try to log in, the login fails.

UPLOAD LEAF IMAGE

The farmer selects the crop and captures or uploads a leaf photograph. The image is checked for format and size and sent to the server for prediction.

VIEW DIAGNOSIS AND ADVICE

The predicted disease, confidence score, symptoms, organic and chemical treatment, dosage and prevention measures are shown. The farmer can play the advice as audio in Tamil.

AGRI OFFICER

VIEW OUTBREAK MAP

Officers view a heat map of diagnoses by village and date range, which helps them identify outbreaks and plan field visits.

VERIFY DIAGNOSIS

Officers can review uncertain predictions and correct the label, which is added to the retraining dataset.

ADMIN

MANAGE KNOWLEDGE BASE

Admin adds, edits and deletes crops, diseases and treatment details used by the advisory engine.

MANAGE USERS

Admin views registered farmers and officers and activates or blocks accounts.

RETRAIN MODEL

Admin uploads new verified images and triggers retraining of the model to improve accuracy.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

The CropCare AI system was implemented as a web-based application with a deep learning back end. The model was trained in a Jupyter notebook on Google Colab and saved in Keras format. The Flask server loads the saved model once at start-up and exposes REST endpoints for prediction, history and administration. The React front end is responsive so that it works on low-cost Android phones.

7.2 Implementation

The dataset consisted of 38 classes from PlantVillage plus around 1,200 field images of paddy and banana leaves collected from nearby farms. Images were split 80:10:10 into training, validation and test sets. Data augmentation (rotation, flip, zoom, brightness change) was applied to make the model robust to field conditions. The MobileNetV2 base was first frozen and the head trained for 10 epochs; then the top 30 layers were unfrozen and fine-tuned with a lower learning rate for another 10 epochs.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.

- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.
- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Login with valid credentials	Registered mobile and password	Dashboard displayed	Pass
TC02	Login with wrong password	Valid mobile, wrong password	Error 'Invalid credentials'	Pass
TC03	Upload non-image file	report.pdf	Error 'Only JPG/PNG allowed'	Pass
TC04	Predict diseased paddy leaf	Leaf blast image	Rice Leaf Blast, confidence > 0.9	Pass
TC05	Predict healthy leaf	Healthy banana leaf	Healthy - no treatment needed	Pass
TC06	Blurred image	Out-of-focus photo	Ask user to retake photo	Pass
TC07	Tamil voice output	Language = Tamil	Tamil MP3 audio plays	Pass
TC08	Outbreak map filter	District = Thanjavur, last 7 days	Heat map with matching points	Pass

PERFORMANCE EVALUATION

The trained model was evaluated on the held-out test set. The following table compares the proposed model with a baseline CNN trained from scratch.

Model	Accuracy	Precision	Recall	F1-Score	Model Size
Simple CNN (scratch)	86.4%	0.85	0.84	0.84	48 MB
VGG16 (transfer)	95.1%	0.95	0.94	0.94	528 MB
MobileNetV2 (proposed)	96.8%	0.97	0.96	0.96	14 MB

7.4 Source Code

train_model.py

```
import tensorflow as tf
from tensorflow.keras import layers, models

IMG = (224, 224)
train = tf.keras.utils.image_dataset_from_directory("dataset/train",
image_size=IMG, batch_size=32)
val = tf.keras.utils.image_dataset_from_directory("dataset/val", image_size=IMG,
batch_size=32)
class_names = train.class_names

augment = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.1),
    layers.RandomBrightness(0.2),
])

base = tf.keras.applications.MobileNetV2(input_shape=IMG + (3,),
include_top=False, weights="imagenet")
base.trainable = False

inputs = layers.Input(shape=IMG + (3,))
x = augment(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(len(class_names), activation="softmax")(x)
model = models.Model(inputs, outputs)

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=
["accuracy"])
model.fit(train, validation_data=val, epochs=10)

# fine-tune the top layers
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(train, validation_data=val, epochs=10)

model.save("models/cropcare_mobilenetv2.keras")
with open("models/classes.txt", "w") as f:
    f.write("\n".join(class_names))
```

app.py

```
import os, uuid, sqlite3
import numpy as np, cv2, tensorflow as tf
from flask import Flask, request, jsonify, send_file
from gtts import gTTS

app = Flask(__name__)
model = tf.keras.models.load_model("models/cropcare_mobilenetv2.keras")
```

```

classes = open("models/classes.txt").read().split("\n")
THRESHOLD = 0.70

def db():
    con = sqlite3.connect("cropcare.db")
    con.row_factory = sqlite3.Row
    return con

def segment_leaf(img):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, (15, 30, 30), (95, 255, 255))
    return cv2.bitwise_and(img, img, mask=mask)

@app.route("/api/predict", methods=["POST"])
def predict():
    f = request.files.get("image")
    if not f or not f.filename.lower().endswith((".jpg", ".jpeg", ".png")):
        return jsonify(error="Only JPG/PNG allowed"), 400
    data = np.frombuffer(f.read(), np.uint8)
    img = cv2.imdecode(data, cv2.IMREAD_COLOR)
    img = cv2.resize(segment_leaf(img), (224, 224))
    x = np.expand_dims(cv2.cvtColor(img, cv2.COLOR_BGR2RGB),
0).astype("float32")
    probs = model.predict(x, verbose=0)[0]
    k = int(np.argmax(probs)); conf = float(probs[k])
    if conf < THRESHOLD:
        return jsonify(status="retake", message="Image unclear. Please take a
closer photo in daylight.")
    lang = request.form.get("lang", "ta")
    con = db()
    dis = con.execute("SELECT * FROM diseases WHERE class_label=?",
(classes[k],)).fetchone()
    tr = con.execute("SELECT type, remedy_en, remedy_ta, dosage FROM treatments
WHERE disease_id=?",
        (dis["disease_id"],)).fetchall()
    advice = [dict(type=t["type"], remedy=t["remedy_ta"] if lang == "ta" else
t["remedy_en"],
        dosage=t["dosage"]) for t in tr]
    audio = f"static/audio/{uuid.uuid4().hex}.mp3"
    gTTS(" ".join(a["remedy"] for a in advice), lang=lang).save(audio)
    con.execute("INSERT INTO diagnosis(user_id, disease_id, confidence,
latitude, longitude, created_at) "
        "VALUES (?, ?, ?, ?, ?, datetime('now'))",
        (request.form.get("user_id"), dis["disease_id"], conf,
        request.form.get("lat"), request.form.get("lng")))
    con.commit()
    return jsonify(status="ok", disease=dis["disease_name"],
confidence=round(conf, 3),
        symptoms=dis["symptoms"], advice=advice, audio="/" + audio)

@app.route("/api/outbreaks")
def outbreaks():
    district = request.args.get("district")
    rows = db().execute("""SELECT d.latitude, d.longitude, s.disease_name FROM
diagnosis d
        JOIN diseases s ON s.disease_id=d.disease_id JOIN
users u ON u.user_id=d.user_id
        WHERE u.district=? AND d.created_at >=
datetime('now', '-7 day')""", (district,)).fetchall()
    return jsonify([dict(r) for r in rows])

```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

7.5 Screen Layout

SCREENSHOTS:

LOGIN

The screenshot shows a web browser window with the URL `https://cropcareai.app/login`. The page has a green header bar with the text "CropCare AI" on the left and navigation links "Home", "Diagnose", "History", "Outbreak Map", and "Admin" on the right. The main content area is light blue and features a white "Farmer Login" form. The form contains two input fields: "Mobile Number" with the value "98765 43210" and "Password" with masked characters "*****". Below the inputs is a green "Login" button. At the bottom of the form, there is a link "New farmer? Register here | தமிழ்".

UPLOAD LEAF IMAGE

https://cropcareai.app/diagnose

CropCare AIHomeDiagnoseHistoryOutbreak MapAdmin

Diagnose Crop Disease

Select Crop

Paddy (பருப்பு)

Leaf Photo

Tap to capture or upload a clear photo of the affected leaf

Language

Tamil

Detect Disease

Tips for a good photo

Take the photo in daylight, not in shade.

Keep only one leaf in the frame.

Hold the phone steady about 20 cm away.

Include the spots or lesions clearly.

DIAGNOSIS RESULT

https://cropcareai.app/

CropCare AIHomeDiagnoseHistoryOutbreak MapAdmin

Diagnosis Result

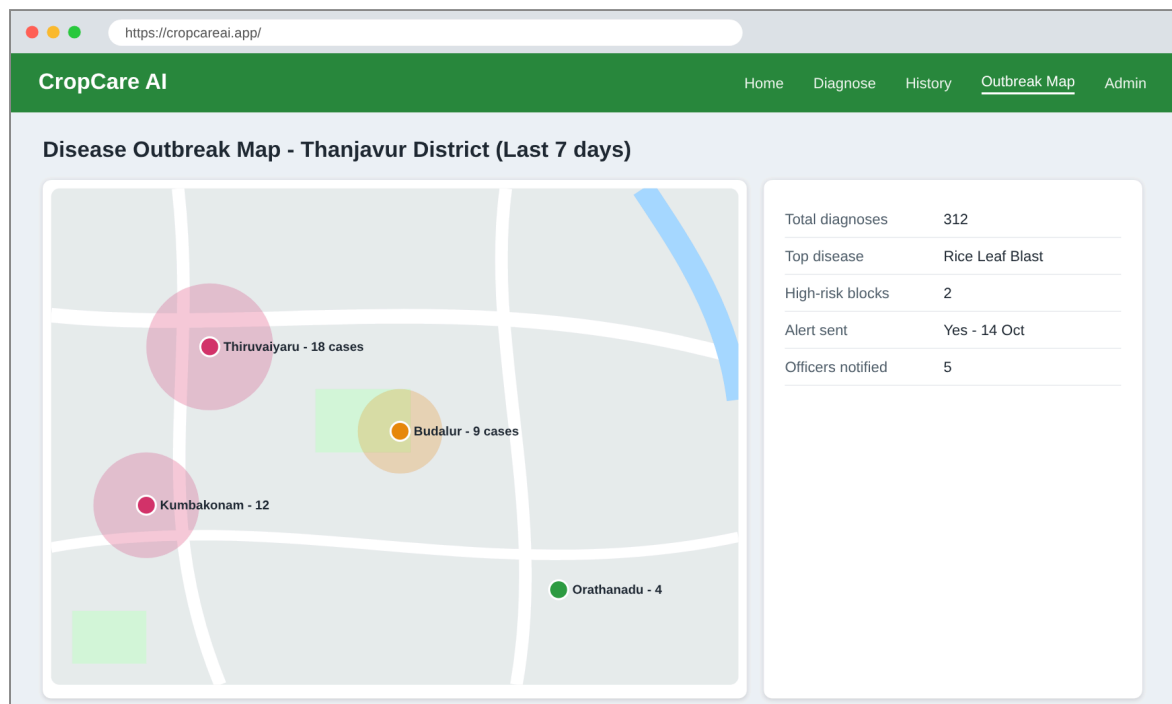
Rice Leaf Blast (Confidence 94.2%)

Crop	Paddy
Pathogen	Magnaporthe oryzae (fungus)
Severity	Moderate
Organic remedy	Pseudomonas fluorescens 10 g / litre spray
Chemical remedy	Tricyclazole 75 WP - 0.6 g / litre
Prevention	Avoid excess nitrogen; keep field bunds clean

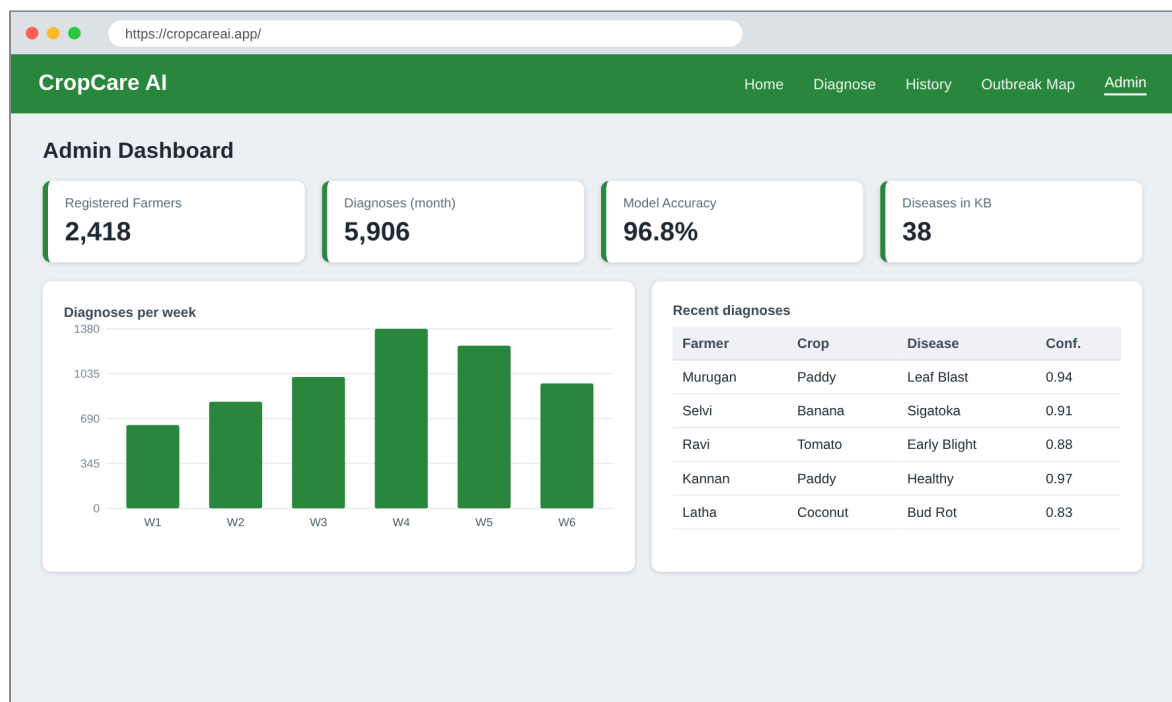
► Play advice in Tamil

Nearest agro centre: Thiruvaiyaru (4.2 km)

OUTBREAK MAP (AGRI OFFICER)



ADMIN DASHBOARD



CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The AI Crop Disease Detection and Advisory System successfully identifies crop diseases from leaf images and provides instant, understandable treatment advice to farmers in Tamil and English. Using transfer learning with MobileNetV2, the model achieved 96.8% accuracy while remaining small enough for fast predictions. The confidence threshold prevents unreliable advice, and the voice output makes the system usable by farmers with limited literacy.

The system reduces dependence on scarce experts, helps farmers use the correct pesticide in the correct dose, and provides the agriculture department with real-time outbreak information. Thus the project addresses a real and urgent problem faced by farmers in delta regions.

8.2 Future Enhancement

- Offline prediction using TensorFlow Lite inside a native Android app.
- Support for more crops and pest (insect) identification.
- Integration with weather data to predict disease risk before symptoms appear.
- Drone imagery for whole-field disease mapping.
- Linking with government input subsidy and crop insurance schemes.

CHAPTER IX

BIBLIOGRAPHY

1. Hughes, D. P., and Salathe, M., "An open access repository of images on plant health to enable the development of mobile disease diagnostics", arXiv:1511.08060, 2015.
2. Mohanty, S. P., Hughes, D. P., and Salathe, M., "Using Deep Learning for Image-Based Plant Disease Detection", *Frontiers in Plant Science*, Vol. 7, Article 1419, 2016.
3. Ferentinos, K. P., "Deep learning models for plant disease detection and diagnosis", *Computers and Electronics in Agriculture*, Vol. 145, pp. 311-318, 2018.
4. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C., "MobileNetV2: Inverted Residuals and Linear Bottlenecks", *Proc. IEEE CVPR*, 2018.
5. Too, E. C., Yujian, L., Njuki, S., and Yingchun, L., "A comparative study of fine-tuning deep learning models for plant disease identification", *Computers and Electronics in Agriculture*, Vol. 161, pp. 272-279, 2019.
6. Chollet, F., *Deep Learning with Python*, 2nd Edition, Manning Publications, 2021.

WEBSITE REFERENCE

1. <https://www.tensorflow.org>
2. <https://flask.palletsprojects.com>
3. <https://agritech.tnau.ac.in>
4. <https://react.dev>